Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Bağlı listenin (linked list) neden gerektiği: eleman sayısının çalışma sırasında sürekli değiştiği durumlar.
- Tek yönlü bağlı liste (singly linked list) düğüm yapısı, `head` ve düğüm oluşturma (`createNode`).
- Başa ekleme (insert at front), sona ekleme (insert at end) ve sıralı listeye araya ekleme (ödev olarak bırakıldı).
- Listeyi baştan sona dolaşıp ekrana yazdırma (display).
- Listeden eleman silme (delete) ve olası durumlar.
- Çift yönlü bağlı liste (doubly linked list): düğüm yapısı, başa ve sona ekleme.
- Dairesel bağlı liste (circular linked list): `last` pointer'ı ile başa ve sona ekleme, listeyi yazdırma.

Hocanın Özellikle Vurguladığı Kısımlar

- Bağlı listede elemanlara sırayla erişilir. Ortadaki bir elemana doğrudan atlanamaz.
- Ekleme yapılacak yerin özel bir önemi yoksa başa ekleyin. Başa ekleme $O(1)$, sona ekleme tek yönlü listede $O(n)$'dir.
- Sona ekleme, sıralamanın önemli olduğu durumlarda yapılır. Örneğin kuyrukta (queue) ilk gelene ilk hizmet verilir.
- `malloc` ile yer ayrıldıktan sonra mutlaka `NULL` kontrolü yapın. Yer ayrılamadıysa üzerinde işlem yapmaya çalışmayın.
- Gereksiz yere global değişken tanımlamayın; hata ayıklamayı zorlaştırır. `head` hemen her fonksiyonda kullanıldığı için bu derste kolaylık olsun diye global tanımlandı. Parametre olarak fonksiyondan fonksiyona aktarmak da mümkündür.
- Araya ekleme ve silmede önceki düğümün adresini (previous) kaybetmemek gerekir.
- Silme işleminde her zaman şu durumlar kontrol edilmeli: liste boş mu, silinecek eleman ilk eleman mı, aradaki bir eleman mı, eleman listede yok mu.
- Algoritma tasarlarken önce kâğıt üzerinde bir örnek çizin. Olası durumları böyle daha kolay görürsünüz.
- Bu derste gösterilen kodları bakmadan kendiniz yazmaya çalışın. Bağlı listeler kuyruk, ağaç ve graf konularında sürekli kullanılacak ve bir daha ayrıntılı anlatılmayacak.
- Hoca hata yaptığında uyarılmak istiyor. Ders sırasında birkaç yazım hatası öğrenciler tarafından fark edildi.
- Online katılım düşük (yaklaşık 90 kayıtlı öğrenciden 16 kişi el kaldırdı). İmkânı olanlar dersi zamanında takip etmeli; kayıtları sonra izlemeyi planlayanlar genelde konuları biriktiriyor.

Kısa Tekrar Notları

- Bellek ayırma seçenekleri:
  - Statik dizi: boyut derleme anında sabit.
  - Dinamik dizi: boyut çalışma başında belli, ardışık blok ayrılır.
  - Bağlı liste: boyut hiç belli değil, eleman geldikçe tek tek yer ayrılır (mail sunucusu, yazıcı kuyruğu gibi).
- Tek yönlü listede son düğümün `next`'i `NULL`'dır.

- Başa ekleme: `newNode->next = head; head = newNode;` $O(1)$.
- Sona ekleme: `next`'i `NULL` olan düğüm bulunur, onun `next`'ine yeni düğüm bağlanır. $O(n)$.
- Dolaşma: `ptr = head; while (ptr != NULL) { ...; ptr = ptr->next; }`
- Silme: önceki düğümün `next`'i, silinen düğümün `next`'ini gösterir; silinen düğüm `free` edilir.
- Çift yönlü listede her düğümde `prev`, `value`, `next` vardır. Önceki düğüm için ayrı değişken tutmaya gerek kalmaz.
- Dairesel listede son düğüm `NULL` yerine ilk düğümü gösterir. `last` tutulursa ilk düğüme `last->next` ile erişilir.
- Dairesel listede başa ve sona ekleme aynı iki satırdır; fark, sona eklemede `last`'ın yeni düğüme taşınmasıdır.
- Dairesel listeyi yazdırmak için `do-while` kullanılır.

Detaylı Açıklamalar

Neden Bağlı Liste?

- **Statik dizi:** En büyük olası boyut verilir veya program her ortam için değiştirilir.
- **Dinamik bellek ayırma:** Eleman sayısı (n) çalışma başında dışarıdan okunur ve o kadar yer ayrılır.
- **Üçüncü durum:** Eleman sayısı çalışma başında da belli değildir. Elemanlar zamanla eklenir ve işi bitince çıkarılır.
  - Mail sunucusu: mailler gelir, bazıları silinir.
  - Yazıcı kuyruğu (printer queue): işler eklenir, biten çıkar.
- Bu durumda en uygun yapı bağlı listedir. Her eleman geldiğinde yalnız o eleman için yer ayrılır. Her düğüm bir sonrakinin adresini tuttuğu için düğümler bellekte dağınık olsa da liste bütünlüğü korunur.

```
head -> [5 | 2000] -> [9 | 2200] -> [7 | NULL]
         (1000)        (2000)        (2200)
```

- Düğümler sadece bir değerden değil, birden fazla alandan (karakter, metin vb.) da oluşabilir. Önemli olan her düğümün bir sonraki düğüme bağlantı tutmasıdır.

Düğüm Yapısı ve Düğüm Oluşturma

```c
#include <stdio.h>
#include <stdlib.h>   /* malloc burada tanımlı */

typedef struct node {
    int value;
    struct node *next;
} node;

node *head;   /* bu derste kolaylık için global */

node *createNode()
{
    node *ptr;
    ptr = (node *) malloc(sizeof(node));
    if (ptr == NULL)
        exit(1);          /* yer ayrılamadı */
    ptr->next = NULL;
    return ptr;
}
```

- `next`, aynı yapıdaki başka bir düğümü gösteren pointer'dır. Hoca bu tanımı "özyinelemeli bir gösterim" diye nitelendirdi.
- Hoca bir önceki derste tahtaya `structure` yazdığını fark etti; doğrusu `struct`'tır.
- `malloc` yer ayıramazsa `NULL` döndürür. Hoca burada programdan `exit` ile çıktı. Gerçek uygulamada bu duruma göre başka bir davranış seçilebilir.

- `createNode` yeni düğümün `next`'ini `NULL` yapar ve düğümün adresini döndürür. Örneğin 9900 adresinde, kimseyi göstermeyen "boşlukta duran" bir düğüm oluşur.
- `main`'de kullanım:

```
int main()
{
    node *ptr;
    head = NULL;               /* başlangıçta liste boş */
    ptr = createNode();
    insertAtFront(ptr, 8);
    ptr = createNode();
    insertAtEnd(ptr, 7);
    ...
}
```

- Fonksiyonlar kullanılmadan önce bildirilmeli (prototype). Hoca bu bildirimlerin, fonksiyonun çağrılacağı yerde (örneğin `main` içinde) yapılmasını önerdi.

Başa Ekleme (Insert at Front)

```
void insertAtFront(node *newNode, int value)
{
    newNode->value = value;
    newNode->next = head;
    head = newNode;
}
```

İzleme:

Adım	İşlem	Liste
Başlangıç	`head = NULL`	(boş)
8 eklenir (9900)	`9900->next = NULL`, `head = 9900`	8
12 eklenir (8500)	`8500->next = 9900`, `head = 8500`	12 → 8
25 eklenir (12000)	`12000->next = 8500`, `head = 12000`	25 → 12 → 8

- Boş liste için ayrı kontrol gerekmez. `head` zaten `NULL` olduğu için yeni düğümün `next`'i `NULL` olur.
- Karmaşıklık $O(1)$'dir.
- Hoca, örnek adreslerin bu kadar uzak olmadığını, bellekte ardışık boş yer varsa düğümlerin genelde yakın yerlere yerleştiğini belirtti. Uzak adresleri, düğümlerin ardışık olmak zorunda olmadığını göstermek için seçti.

Sona Ekleme (Insert at End)

```
void insertAtEnd(node *newNode, int value)
{
    node *temp;
    newNode->value = value;
    if (head == NULL)              /* liste boş */
        head = newNode;
    else {                         /* son düğümü bul */
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
}
```

Örnek liste: `head -> 1000 [7] -> 1200 [5] -> 1400 [9] -> NULL`, yeni düğüm 1600 adresinde, değeri 27:

temp	temp->next	Karar
1000	1200	NULL değil, ilerle
1200	1400	NULL değil, ilerle
1400	NULL	Dur, `1400->next = 1600`

- Son düğüm, `next`'i `NULL` olan düğümdür.
- Liste boşsa `head` doğrudan yeni düğümü gösterir.
- Karmaşıklık $O(n)$'dir.
- Bir öğrenci `head`'in fonksiyona parametre olarak gelmediğini sordu. Cevap: `head` global tanımlandığı için tüm fonksiyonlar ona erişebilir.

Sıralı Listeye Ekleme (Ödev)

- Hoca bu kısmı öğrencilerin kendisinin yazmasını istedi.
- Örnek: küçükten büyüğe sıralı `5 → 7 → 11` listesine 1400 adresindeki 8 eklenecek.
- Yöntem:
  - 8, sırayla 5, 7 ve 11 ile karşılaştırılır.
  - 11'e gelindiğinde 8'in 11'den küçük olduğu anlaşılır; 8, 7 ile 11 arasına girmelidir.
  - 7'nin düğümü (1200) artık 1400'ü, 1400 de 11'in düğümünü (1300) göstermelidir.
- Dikkat: 11'e gelindiğinde 7'nin adresi kaybedilmiştir. Bu yüzden bir önceki düğümün adresi ayrı bir değişkende tutulmalıdır.

Listeyi Yazdırma (Display)

```c
void display()
{
    node *ptr = head;
    while (ptr != NULL) {
        printf("%d ", ptr->value);
        ptr = ptr->next;
    }
}
```

- Adım sayısı belli olmayan bir döngüdür; `ptr` `NULL` olana kadar devam edilir.
- `head -> 1000 [5] -> 1200 [9] -> NULL` için ekrana `5 9` yazılır.

Eleman Silme (Delete)

Olası durumlar:

1. **Liste boş** (`head == NULL`): Silinecek bir şey yok, çıkılır.
2. **Silinecek eleman ilk eleman:** `head` değişmelidir. `head`, ikinci düğümü göstermelidir.
3. **Silinecek eleman aradaki veya sondaki bir eleman:** Önceki düğüm, silinen düğümün sonrasını göstermelidir. Bunun için önceki düğümün adresi tutulmalıdır.
4. **Eleman listede yok:** Liste sonuna kadar gidilir, "not found" verilir.

```c
void deleteNode(int del)
{
    node *temp, *prev, *current;
    if (head == NULL)
        return;                              /* liste boş */
    if (head->value == del) {                /* ilk eleman */
        temp = head;
        head = head->next;
        free(temp);
    }
    else {
```

```
        prev = head;
        current = head->next;
        while (current != NULL && current->value != del) {
            prev = current;
            current = current->next;
        }
        if (current == NULL)
            printf("not found");
        else {
            prev->next = current->next;
            free(current);
        }
    }
}
```

Örnek liste: `head -> 1000 [7] -> 1200 [5] -> 1300 [11] -> NULL`

- **7 silinirse:** `temp = 1000`, `head = 1200`, `free(1000)`. 1000 adresi artık başkaları tarafından kullanılabilir.
- **11 silinirse:**

prev	current	current->value	Karar
1000	1200	5	11 değil, ilerle
1200	1300	11	Bulundu, döngüden çık

- `prev->next = current->next` ile 1200'ün `next`'i `NULL` olur. `free(1300)`.
- **Liste yalnız `[5]` iken 8 silinmeye çalışılırsa:** `prev = head`, `current = NULL`. Döngüye girilmez, "not found" yazılır.
- Döngüden iki sebeple çıkılır: Ya liste bitmiştir (`current == NULL`) ya da eleman bulunmuştur. Bu yüzden döngüden sonra hangisi olduğu kontrol edilir.

Çift Yönlü Bağlı Liste (Doubly Linked List)

- Tek yönlü listede araya ekleme ve silmede önceki düğümü ayrı bir değişkende tutmak gerekiyordu. Çift yönlü liste bu ihtiyacı ortadan kaldırmak için tasarlanmıştır.
- Her düğüm hem sonrakini (`next`) hem öncekini (`prev`) gösterir.
- İlk düğümün `prev`'i, son düğümün `next`'i `NULL`'dır. Listenin başına front, sonuna end denir.

```
NULL <- [prev | 9 | next] <-> [prev | 7 | next] <-> [prev | 8 | next] -> NULL
             (1000)               (1200)               (1300)
```

```
typedef struct node {
    struct node *prev;
    int value;
    struct node *next;
} node;

node *head;

node *createNode()
{
    node *ptr = (node *) malloc(sizeof(node));
    if (ptr == NULL)
        exit(1);
    ptr->next = NULL;
    ptr->prev = NULL;
    return ptr;
}
```

Başa Ekleme

```c
void insertAtFront(node *newNode, int value)
{
    newNode->value = value;
    if (head == NULL)
        head = newNode;
    else {
        head->prev = newNode;
        newNode->next = head;
        head = newNode;
    }
}
```

- Örnek: `head -> 1500 [5] <-> 1800 [2]`, yeni düğüm 1700, değeri 12.
  - `1500->prev = 1700`.
  - `1700->next = 1500`.
  - `head = 1700`.
- Sonuç: `head -> 1700 [12] <-> 1500 [5] <-> 1800 [2]`. 1700'ün `prev`'i `NULL` kalır.

Sona Ekleme

```c
void insertAtEnd(node *newNode, int value)
{
    node *temp;
    newNode->value = value;
    if (head == NULL)
        head = newNode;
    else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
}
```

- Örnek: `head -> 1500 [9] <-> 1700 [8]`, yeni düğüm 1900.
  - `temp` 1500'den başlar, 1700'e ilerler. 1700'ün `next`'i `NULL` olduğu için durur.
  - `1700->next = 1900`, `1900->prev = 1700`.
- Tek yönlü listeden tek farkı, yeni düğümün `prev`'inin de bağlanmasıdır.

Dairesel Bağlı Liste (Circular Linked List)

- Son düğüm `NULL` yerine ilk düğümü gösterir. Kuyruk yapılarında çok kullanılır.
- Düğüm yapısı tek yönlü listeyle aynıdır (`value`, `next`).
- `head` yerine son düğümü gösteren `last` tutulabilir. İlk düğüme `last->next` ile ulaşılır. Başlangıçta `last = NULL`'dır.

```
    +-------------------------------------+
    v                                     |
 [3 | 1200] -> [4 | 1400] -> [11 | 1000] -+
   (1000)        (1200)        (1400)  <- last
```

Sona Ekleme

```c
void insertAtEnd(node *newNode, int value)
{
    newNode->value = value;
    if (last == NULL) {             /* liste boş */
        last = newNode;
        newNode->next = last;       /* kendini gösterir */
    }
```

```
    else {
        newNode->next = last->next; /* yeni düğüm ilk düğümü gösterir */
        last->next = newNode;       /* eski son düğüm yeni düğümü gösterir */
        last = newNode;             /* yeni düğüm artık son düğüm */
    }
}
```

- Liste boşsa tek düğüm kendini gösterir.
- Örnek: `last = 1400`, liste `3 → 4 → 11 → (3)`, yeni düğüm 1700, değeri 15.
  - `1700->next = 1400->next = 1000`.
  - `1400->next = 1700`.
  - `last = 1700`.
- Sonuç: `3 → 4 → 11 → 15 → (3)`.
- Hoca bunu başa ekleme olarak yazmaya başladı, `last = newNode` satırıyla sona ekleme olduğunu fark edip düzeltti.

Başa Ekleme

```
void insertAtFront(node *newNode, int value)
{
    newNode->value = value;
    if (last == NULL) {
        last = newNode;
        newNode->next = last;
    }
    else {
        newNode->next = last->next;
        last->next = newNode;       /* last değişmez */
    }
}
```

- Örnek: `last = 1200`, liste `1000 [9] → 1200 [8] → (1000)`, yeni düğüm 1300, değeri 12.
  - `1300->next = 1200->next = 1000`.
  - `1200->next = 1300`.
  - `last` hâlâ 1200'dür.
- Sonuç: `12 → 9 → 8 → (12)`. 12 listenin başı olur.
- Başa ve sona ekleme aynı iki bağlantı işlemini yapar. Fark yalnız `last`'ın güncellenip güncellenmemesidir.

Dairesel Listeyi Yazdırma Hocanın ilk yazdığı döngü:

```
for (ptr = last->next; ptr != last->next; ptr = ptr->next)
    printf("%d ", ptr->value);
```

- Bir öğrenci bu döngüye hiç girilmeyeceğini fark etti. `ptr` `last->next` ile başlatılıyor ve ilk kontrolde koşul hemen yanlış oluyor.
- Hoca öğrenciye teşekkür edip kontrolü sona alan `do-while` ile düzeltti:

```
void display()
{
    node *ptr;
    if (last == NULL)
        printf("list is empty");
    else {
        ptr = last->next;           /* ilk düğüm */
        do {
            printf("%d ", ptr->value);
            ptr = ptr->next;
        } while (ptr != last->next);
    }
}
```

- Örnek: `300 [11] → 500 [7] → 900 [8] → (300)`, `last = 900`.

`ptr`	Yazılan	Sonraki `ptr`	`ptr != 300`?
300	11	500	Evet, devam
500	7	900	Evet, devam
900	8	300	Hayır, dur

- Ekrana `11 7 8` yazılır.

Ders sonrası notlar

- **Fonksiyon bildirimlerinin yeri:** Hoca, fonksiyon prototiplerinin çağrıldıkları fonksiyonun içinde (örneğin `main` içinde) yapılmasının doğru olduğunu, dışarıda yapılmasının kafa karıştırıcı olduğunu söyledi. C'de yaygın ve önerilen uygulama prototipleri dosya başında (global kapsamda) veya bir başlık (header) dosyasında yazmaktır. Blok içinde bildirim de geçerlidir ama nadiren kullanılır. Sınav ve ödevlerde hocanın tercihini sormak faydalı olabilir.
- **`malloc`'un başarısız olması:** Hoca bunu "yer yoksa veya salt okunur bir bellek bölgesiyle ilgili işlem yapılıyorsa" diye açıkladı. `malloc` pratikte heap'te istenen boyutta yer ayrılamadığında `NULL` döndürür. Salt okunur bölgeye yazmak ise `malloc` hatası değil, çalışma zamanı hatasıdır (segmentation fault).
- **Dairesel listede başa eklemenin açıklaması:** Hoca başa ekleme kodunu bitirince "elemanı dizinin sonuna yerleştirmiş oluyoruz" dedi. Bu bir dil sürçmesidir; `last` değişmediği için eleman listenin başına eklenir.
- **Silmede `free`:** Hoca "kimse 9'u göstermeyecek, `free` ayrı konu" dedi. Kodda bağlantıyı koparmakla birlikte `free` çağrısı da yapılmalıdır; aksi hâlde bellek sızıntısı (memory leak) olur.
- **Başa eklemede `head = newNode`:** Hoca tahtada üçüncü satırı yanlış bir işaretle yazdı; bir öğrenci uyardı. Doğrusu atama işlemidir: `head = newNode;`.
- **`createNode` içindeki değişken adı:** Hoca bir yerde `p`, bir yerde `ptr` yazdı ve düzeltti. Değişken adları tutarlı olmalıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.